

RISC-V M 模式 特权架构详解

面向 RT-Thread 移植：yield 软中断与上下文切换

基于 RISC-V Privileged ISA Spec V20211203

本文档专为「一生一芯」C 阶段 RT-Thread 移植而编写。
在纯 M 模式下运行 RT-Thread，没有硬件定时器中断，
靠 yield 自陷软中断实现线程调度与上下文切换。
本文档从 ISA 定义到硬件实现到软件编程，
逐一拆解上下文切换发生的全部事情。

目录

I M 模式指令集架构定义	5
1 M 模式总览	5
1.1 M 模式在一生一芯中的角色	5
1.2 M 模式的最小实现要求	5
2 M 模式核心 CSR 详解	6
2.1 mstatus — 机器状态寄存器	6
2.2 mie / mip — 中断使能与挂起	6
2.3 mtvec — 陷阱向量基址寄存器	7
2.4 mscratch — 暂存寄存器	7
2.5 mepc / mcause / mtval — 陷阱上下文三件套	7
2.6 mhartid — 硬件线程编号	7
3 M 模式特权指令	7
3.1 ECALL — 环境调用 (yield 的核心)	7
3.2 MRET — 机器级异常返回	8
3.3 EBREAK — 断点	8
3.4 WFI — 等待中断	8
4 M 模式陷阱的完整定义	8
4.1 陷阱触发条件	8
4.2 陷阱进入时的硬件行为 (精确时序)	9
4.3 MRET 时的硬件行为	9
4.4 陷阱与中断的区别	10
II yield 软中断与上下文切换全流程	10
5 yield 机制的整体架构	10
5.1 为什么需要 yield	10
5.2 yield 的整体流程	10
5.3 线程控制块 (TCB) 与栈的关系	11
6 上下文保存与恢复的完整拆解	11
6.1 需要保存的寄存器	11
6.2 栈帧布局	12
6.3 陷阱入口代码 (保存上下文)	12
6.4 mepc 加 4 的问题	16

7 分情况讨论：上下文切换的各种场景	16
7.1 场景一：线程 A 首次 yield，切换到线程 B	16
7.2 场景二：线程 B 首次运行（从新创建的线程启动）	16
7.3 场景三：idle 线程的 yield	17
7.4 场景四：线程被阻塞后恢复	18
7.5 场景五：中断嵌套（如果有硬件中断）	18
8 yield 的完整实现代码	18
8.1 汇编入口（trap_entry）	18
8.2 C 语言调度器	21
8.3 yield 函数	21
8.4 线程启动	22
9 常见陷阱与调试技巧	22
9.1 陷阱一：忘记 mepc 加 4	22
9.2 陷阱二：mscratch 使用错误	22
9.3 陷阱三：栈对齐问题	22
9.4 陷阱四：gp 寄存器未正确设置	22
9.5 陷阱五：调度器栈溢出	23
9.6 调试技巧	23
III RT-Thread 移植实践	23
10 RT-Thread 的架构与移植要点	23
10.1 RT-Thread Nano 的组件	23
10.2 yield 方案与标准方案的区别	24
11 移植的完整步骤	24
11.1 步骤一：实现 rt_hw_stack_init	24
11.2 步骤二：实现 rt_hw_context_switch	25
11.3 步骤三：实现 rt_hw_context_switch_to	27
11.4 步骤四：实现 rt_hw_exception	28
11.5 步骤五：连接到 RT-Thread 调度器	29
12 从启动到第一个线程运行	29
12.1 启动流程	29
12.2 rtthread_startup	30
12.3 rt_system_scheduler_start	30
12.4 第一个线程的运行	31

13 进阶：从 yield 到定时器中断调度	31
13.1 为什么需要定时器中断	31
13.2 实现 CLINT 定时器中断	31
13.3 定时器中断处理	32
13.4 yield 与定时器中断的共存	33
14 完整示例：两个线程交替运行	33
14.1 main.c	33
14.2 执行流程	34
14.3 验证方法	34

Part I

M 模式指令集架构定义

本部分从 ISA 规范的角度，完整定义 RISC-V M 模式（Machine Mode）的指令集架构。M 模式是 RISC-V 中权限最高的模式——处理器复位后总是从 M 模式启动，M 模式可以访问所有 CSR、执行所有特权指令、直接控制全部硬件资源。在「一生一芯」的 RT-Thread 移植场景中，整个系统运行在 M 模式下，没有 S 模式和 U 模式，因此 M 模式的 ISA 就是全部的特权架构。

1 M 模式总览

1.1 M 模式在一生一芯中的角色

在标准 RISC-V 系统中，M 模式运行固件（如 OpenSBI），S 模式运行操作系统，U 模式运行用户程序。但在「一生一芯」C 阶段的 RT-Thread 移植中，情况完全不同——整个 RT-Thread 内核和所有线程都运行在 M 模式下，没有特权级切换。这种设计简化了实现（不需要委托、不需要 PMP 隔离），但也带来了一个核心挑战：**如何在没有硬件定时器中断的情况下实现线程调度？**

答案是 `yield`——一种基于 `ECALL` 自陷的软件中断机制。线程主动调用 `yield` 时，通过 `ECALL` 触发 M 模式陷阱，陷阱处理程序保存当前线程上下文、选择下一个线程、恢复其上下文、执行 `MRET` 返回。整个过程不依赖任何外部中断源，完全由软件驱动。

1.2 M 模式的最小实现要求

对于 RT-Thread 移植，M 模式需要实现的最小 CSR 集合：

地址名称用途 RT-Thread 中的角色			
0x300	<code>mstatus</code>	机器状态	保存/恢复 MPIE、MIE
0x304	<code>mie</code>	中断使能	使能 MSIE（软件中断）
0x305	<code>mtvec</code>	陷阱向量	指向 <code>yield</code> 陷阱入口
0x340	<code>mscratch</code>	暂存	保存当前线程栈指针
0x341	<code>mepc</code>	异常 PC	保存/恢复 <code>yield</code> 调用点
0x342	<code>mcause</code>	异常原因	区分 <code>ECALL</code> （ <code>cause=11</code> ）与其他
0x343	<code>mtval</code>	附加信息	异常地址（RT-Thread 中少用）
0x344	<code>mip</code>	中断挂起	读取 MSIP 状态
0xF14	<code>mhartid</code>	Hart ID	多核识别（单核固定为 0）

关键点：在纯 M 模式 RT-Thread 中，`mstatus.MIE` 的作用与标准 OS 不同。标准 OS 中 MIE 控制全局中断；在 RT-Thread `yield` 方案中，MIE 可以始终为 1（因为 `ECALL` 是同步异常，不受 MIE 控制），但需要正确管理 MPIE 以支持嵌套陷阱（虽然 RT-Thread 通常不需要嵌套）。

2 M 模式核心 CSR 详解

2.1 mstatus — 机器状态寄存器

mstatus（地址 0x300）是 M 模式最重要的状态寄存器。在 RT-Thread yield 方案中，我们只关心以下几个位：

位名称含义 yield 中的作用			
3	MIE	全局 M 中断使能	陷阱进入时硬件清 0，MRET 时从 MPIE 恢复
7	MPIE	陷阱前的 MIE 旧值	保存陷阱前的 MIE，MRET 时恢复
12:11	MPP	陷阱前的特权级	纯 M 模式下始终为 11 (M)，MRET 时恢复

陷阱进入时硬件自动执行：

1. $MPIE \leftarrow MIE$ （保存旧值）
2. $MIE \leftarrow 0$ （关闭中断，防止嵌套）
3. $MPP \leftarrow$ 当前特权级（纯 M 模式下 = 11）
4. $mepc \leftarrow$ 触发指令的 PC
5. $mcause \leftarrow$ 异常原因码
6. $PC \leftarrow mtvec$

MRET 执行时硬件自动执行：

1. $MIE \leftarrow MPIE$ （恢复中断使能）
2. $MPIE \leftarrow 1$
3. $MPP \leftarrow 11$ （M 模式下）
4. 特权级 $\leftarrow MPP$
5. $PC \leftarrow mepc$

易错点：ECALL 是同步异常，不受 MIE 控制——即使 $MIE=0$ ，ECALL 仍然会触发陷阱。MIE 只控制异步中断（MSI/MTI/MEI）。这意味着 yield 机制不需要依赖 MIE 的状态，但陷阱处理程序中仍需正确管理 MIE/MPIE，以防万一。

2.2 mie / mip — 中断使能与挂起

mie（0x304）和 mip（0x344）是中断使能和挂起寄存器。在 yield 方案中，它们的主要角色是支持 **软件中断（MSI）**：

在纯 yield 方案中，ECALL 是同步异常，不经过 mie/mip。但如果你的处理器实现了 CLINT 定时器中断，RT-Thread 也可以用定时器中断驱动调度（更接近真实 OS）。本文档主要讨论 yield 方案，定时器中断方案作为进阶在第三部分讨论。

位名称含义说明

3	MSIE / MSIP	M 级软件中断	写 CLINT 的 MSIP 寄存器可触发
7	MTIE / MTIP	M 级定时器中断	写 CLINT 的 mtimecmp 可触发
11	MEIE / MEIP	M 级外部中断	PLIC 驱动

2.3 mtvec — 陷阱向量基址寄存器

mtvec (0x305) 告诉硬件「发生陷阱时跳到哪里」。它由两部分组成：

- **BASE** (位 31:2)：陷阱入口地址，必须 4 字节对齐。
- **MODE** (位 1:0)：0 = Direct (所有陷阱跳到同一地址)，1 = Vectored (中断按 cause 向量化)。

RT-Thread yield 方案使用 Direct 模式——所有陷阱 (ECALL、非法指令、地址对齐错误等) 都跳到同一个入口，由软件读 mcause 分发。

Listing 1: 设置 mtvec

```
1  la t0, trap_entry      # 加载陷阱入口地址
2  csrw mtvec, t0         # 写入 mtvec
```

2.4 mscratch — 暂存寄存器

mscratch (0x340) 是一个纯粹的 32 位通用读写寄存器，硬件不对其做任何特殊处理。在 yield 方案中，它扮演关键角色：**保存当前运行线程的栈指针**。

为什么需要 mscratch？因为陷阱入口处，我们需要一个地方保存当前 sp，然后切换到调度器栈。通用寄存器在陷阱入口时还保存着线程的值，不能直接用；CSR 寄存器中只有 mscratch 是「自由」的（不被硬件自动修改）。

2.5 mepc / mcause / mtval — 陷阱上下文三件套

这三个寄存器在陷阱进入时由硬件自动写入：

mepc 的关键细节：对于 ECALL 异常，mepc 指向 ECALL 指令本身。如果直接 MRET，会重新执行 ECALL，导致无限循环。因此，陷阱处理程序必须在 MRET 前将 mepc 加 4（跳过 ECALL 指令）。

2.6 mhartid — 硬件线程编号

mhartid (0xF14) 是只读寄存器，返回当前 hart 的编号。单核系统中固定为 0。RT-Thread 在初始化时读取它来确认 hart ID。

3 M 模式特权指令

3.1 ECALL — 环境调用 (yield 的核心)

ECALL 指令同步触发一个「环境调用」异常。在 M 模式下执行 ECALL，mcause 被设为 11 (environment call from M-mode)，PC 跳到 mtvec。

寄存器地址陷阱时写入 MRET 时用途

mepc	0x341	触发陷阱的指令地址	PC ← mepc (返回)
mcause	0x342	原因码（最高位 = 中断/异常，低 31 位 = 编号）	软件 读取 分发
mtval	0x343	附加信息（异常地址、非法指令编码等）	软件 读取 诊断

yield 的本质：RT-Thread 的 `yield()` 函数底层就是执行 ECALL。线程调用 `yield` → ECALL 触发陷阱 → 陷阱处理程序执行调度 → MRET 返回到另一个线程。这就是「软中断」——不是硬件中断，而是软件主动触发的同步异常。

3.2 MRET —机器级异常返回

MRET 是 M 模式的陷阱返回指令。执行时硬件自动恢复 `mstatus` ($MIE \leftarrow MPIE$, $MPIE \leftarrow 1$) 并将 PC 设为 `mepc`。在 `yield` 方案中，MRET 是上下文切换的最后一步——它将 CPU 控制权交给被选中的线程。

3.3 EBREAK —断点

EBREAK 触发 `cause=3` 的异常。在 RT-Thread 中少用，但调试器可能使用。

3.4 WFI —等待中断

WFI 暂停 hart 执行，直到有中断到来。在纯 `yield` 方案中，如果没有硬件中断，WFI 会永久停机——因此 RT-Thread idle 线程通常不使用 WFI，而是循环调用 `yield`。

4 M 模式陷阱的完整定义

4.1 陷阱触发条件

在纯 M 模式 RT-Thread 中，可能触发的陷阱：

cause 类型说明 yield 方案中的频率

2	异常	非法指令	罕见（调试时可能）
3	异常	断点（EBREAK）	罕见
4	异常	Load 地址对齐错误	偶发（指针错误）
5	异常	Load 访问错误	偶发
6	异常	Store/AMO 地址对齐错误	偶发
7	异常	Store/AMO 访问错误	偶发
11	异常	ECALL from M-mode	每次 yield

4.2 陷阱进入时的硬件行为（精确时序）

当 ECALL 执行时，硬件在**同一条指令内**完成以下操作（不需要额外的时钟周期）：

1. $mepc \leftarrow \text{ECALL 指令的 PC 地址}$
2. $mcause \leftarrow 11$ （M 模式 ECALL）
3. $mstatus.MPIE \leftarrow mstatus.MIE$
4. $mstatus.MIE \leftarrow 0$
5. $mstatus.MPP \leftarrow 11$ （当前是 M 模式）
6. $PC \leftarrow mtvec.BASE$ （Direct 模式）

注意：通用寄存器（x0–x31）**不被硬件修改**——它们保持 ECALL 执行前的值。保存通用寄存器是软件（陷阱处理程序）的职责。

4.3 MRET 时的硬件行为

MRET 执行时，硬件完成：

1. $mstatus.MIE \leftarrow mstatus.MPIE$
2. $mstatus.MPIE \leftarrow 1$
3. $mstatus.MPP \leftarrow 11$ （M 模式下）
4. $PC \leftarrow mepc$

同样，通用寄存器不被硬件修改——恢复通用寄存器是软件的责任。

4.4 陷阱与中断的区别

关键区别: ECALL 是同步异常(synchronous exception),不是异步中断(asynchronous interrupt)。同步异常在指令执行时产生, PC 指向触发指令本身; 异步中断在指令之间产生, PC 指向下一条指令。这意味着:

- ECALL 不受 mstatus.MIE 控制——即使 MIE=0, ECALL 仍触发陷阱。
- mepc 指向 ECALL 指令本身, 不是下一条指令。陷阱处理程序必须 mepc += 4。
- mcause 最高位为 0 (异常), 不是 1 (中断)。

Part II

yield 软中断与上下文切换全流程

本部分是文档的核心。我们将从零开始, 逐步构建一个完整的 yield 上下文切换机制。每一节都会拆解上下文切换过程中发生的全部事情, 分情况讨论各种边界条件, 并给出可编译运行的汇编代码。

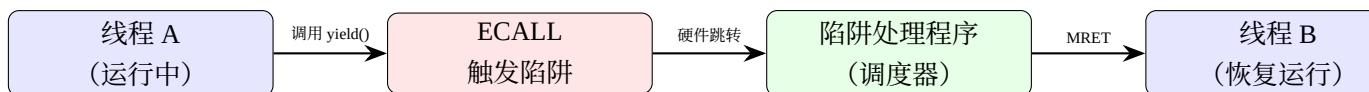
5 yield 机制的整体架构

5.1 为什么需要 yield

标准 RTOS (如 FreeRTOS、RT-Thread) 通常依赖硬件定时器中断实现时间片调度——定时器每隔 1ms 触发中断, 中断处理程序调用调度器切换线程。但在「一生一芯」C 阶段, 学生可能还没有实现 CLINT 定时器中断, 或者定时器中断不够稳定。此时, **协作式调度** (cooperative scheduling) 是一个可行的替代方案——线程主动调用 yield() 放弃 CPU, 调度器选择下一个线程运行。

yield 的底层实现就是 ECALL——一条指令触发陷阱, 陷阱处理程序完成上下文切换。这种方案不需要任何外部中断源, 只需要处理器正确实现 ECALL 异常和 MRET 返回指令。

5.2 yield 的整体流程



保存 A 的上下文 → 选择 B → 恢复 B 的上下文

详细步骤:

1. 线程 A 执行 yield(), 底层是 ECALL 指令。
2. 硬件检测到 ECALL, 保存 mepc/mcause/mstatus, 跳转到 mtvec。

3. 陷阱入口代码（汇编）保存线程 A 的通用寄存器到 A 的栈上。
4. 调度器（C 函数）选择下一个线程 B。
5. 陷阱出口代码（汇编）从 B 的栈恢复通用寄存器。
6. 执行 MRET，硬件恢复 mepc/mstatus，PC 跳到 B 上次 yield 的下一条指令。

5.3 线程控制块（TCB）与栈的关系

每个线程有一个线程控制块（TCB）和一个独立的栈。上下文切换的核心是**保存当前线程的寄存器到它的栈上，从下一个线程的栈恢复寄存器**。

Listing 2: 线程控制块结构

```
1 struct thread {
2     uint32_t *sp;           // 栈指针（指向保存的上下文）
3     void (*entry)(void *); // 线程入口函数
4     void *arg;              // 入口参数
5     uint8_t *stack;         // 栈底地址
6     uint32_t stack_size;    // 栈大小
7     enum { READY, RUNNING, BLOCKED } state;
8 };
```

sp 字段是上下文切换的关键——它指向线程被切换出去时，栈顶保存的寄存器区域。

6 上下文保存与恢复的完整拆解

6.1 需要保存的寄存器

RISC-V RV32IM 有 32 个通用寄存器（x0–x31）。上下文切换时需要保存哪些？

寄存器名称调用者保存？上下文切换中的处理

x0	zero	—	常量 0，不需要保存
x1	ra	调用者	必须保存 （返回地址）
x2	sp	被调用者	必须保存 （栈指针）
x3	gp	—	通常不保存（全局指针固定）
x4	tp	—	通常不保存（线程指针, RT-Thread 可能用）
x5–x7	t0–t2	调用者	必须保存
x8	s0/fp	被调用者	必须保存
x9	s1	被调用者	必须保存
x10–x11	a0–a1	调用者	必须保存
x12–x17	a2–a7	调用者	必须保存
x18–x27	s2–s11	被调用者	必须保存
x28–x31	t3–t6	调用者	必须保存

为什么全部保存？ 标准 RISC-V ABI 规定调用者保存寄存器（ra, t0–t6, a0–a7）由调用者负责保存，被调用者保存寄存器（sp, s0–s11）由被调用者负责保存。但在上下文切换中，我们不知道线程在 ECALL 时正在执行什么代码——它可能在一个函数的中间，所有寄存器都是「活跃」的。因此，**必须保存除 x0（zero）、gp、tp 之外的所有寄存器。**

实际保存的寄存器列表（30 个）：ra, sp, t0–t6, s0–s11, a0–a7。加上 mepc 和 mstatus（需要手动保存，因为 MRET 会修改它们），总共 32 个字（128 字节）。

6.2 栈帧布局

当线程被切换出去时，它的栈顶布局如下（从高地址到低地址）：

Listing 3: 上下文在栈上的布局

```

1 # 高地址（栈底方向）
2 #   ... 线程之前的栈内容 ...
3 #
4 #   mstatus           ← 保存的 mstatus
5 #   mepc              ← 保存的 mepc（ECALL 的下一条指令）
6 #   ra                ← x1
7 #   sp                ← x2（保存的是线程自己的 sp）
8 #   t0                ← x5
9 #   t1                ← x6
10 #   ...              ← ...
11 #   t6                ← x31
12 #   s0                ← x8
13 #   ...              ← ...
14 #   s11              ← x27
15 #   a0                ← x10
16 #   ...              ← ...
17 #   a7                ← x17
18 #   ↓ sp 指向这里    ← thread.sp 保存这个地址
19 #
20 # 低地址（栈顶方向）

```

6.3 陷阱入口代码（保存上下文）

以下是完整的陷阱入口汇编代码，逐行注释：

Listing 4: 陷阱入口：保存上下文

```

1 trap_entry:
2   # 1. 交换 sp 和 mscratch
3   #   此时 sp 是线程 A 的栈指针
4   #   mscratch 保存的是调度器栈指针（或上一个线程的 sp）
5   csrrw sp, mscratch, sp   # sp ↔ mscratch
6   # 现在 sp = 调度器栈，mscratch = 线程A的sp
7

```

```

8      # 2. 在线程 A 的栈上分配空间保存寄存器
9      #      需要 32 个字 = 128 字节
10     addi    sp, sp, -128          # 分配 128 字节
11
12     # 3. 保存通用寄存器 (使用 mscratch 中的线程 A 的 sp)
13     #      注意: 此时 sp 是调度器栈, 不是线程 A 的栈!
14     #      我们需要用 mscratch 中的地址来保存到线程 A 的栈。
15     #      但更简单的做法是: 先在调度器栈上保存, 再拷贝。
16     #      或者: 直接用 mscratch 作为基址保存。
17     #
18     #      实际上, 更常见的做法是:
19     #      交换后, sp 变成线程栈, mscratch 变成调度器栈。
20     #      然后在线程栈上保存寄存器。
21
22     # 重新设计: 交换后 sp = 线程A的栈
23     # 让我们重新来:
24
25     trap_entry:
26         # sp = 线程 A 的栈指针
27         # mscratch = 调度器栈指针 (上次设置的)
28
29         # 1. 在线程 A 的栈上分配 128 字节
30         addi    sp, sp, -128
31
32         # 2. 保存所有通用寄存器
33         sw      ra,    0(sp)
34         sw      t0,    4(sp)
35         sw      t1,    8(sp)
36         sw      t2,   12(sp)
37         sw      s0,   16(sp)
38         sw      s1,   20(sp)
39         sw      a0,   24(sp)
40         sw      a1,   28(sp)
41         sw      a2,   32(sp)
42         sw      a3,   36(sp)
43         sw      a4,   40(sp)
44         sw      a5,   44(sp)
45         sw      a6,   48(sp)
46         sw      a7,   52(sp)
47         sw      s2,   56(sp)
48         sw      s3,   60(sp)
49         sw      s4,   64(sp)
50         sw      s5,   68(sp)
51         sw      s6,   72(sp)
52         sw      s7,   76(sp)

```

```

53      SW      s8, 80(sp)
54      SW      s9, 84(sp)
55      SW      s10, 88(sp)
56      SW      s11, 92(sp)
57      SW      t3, 96(sp)
58      SW      t4, 100(sp)
59      SW      t5, 104(sp)
60      SW      t6, 108(sp)
61
62      # 3. 保存 mepc 和 mstatus (MRET 会修改它们)
63      csrr      t0, mepc
64      SW      t0, 112(sp)
65      csrr      t0, mstatus
66      SW      t0, 116(sp)
67
68      # 4. 保存线程 A 的 sp 到 TCB
69      #      sp + 128 = 线程 A 保存上下文前的栈指针
70      #      但我们需要保存的是「分配 128 字节后的 sp」
71      #      即当前 sp 的值——这样恢复时直接用这个 sp
72      addi     t0, sp, 0          # t0 = 当前 sp (指向保存的上下文)
73      la       t1, current_thread # t1 = &current_thread
74      lw       t1, 0(t1)         # t1 = current_thread (TCB 指针)
75      sw       t0, 0(t1)         # current_thread->sp = t0
76
77      # 5. 切换到调度器栈
78      csrr      sp, mscratch     # sp = 调度器栈
79
80      # 6. 调用 C 语言调度器
81      call      schedule         # 选择下一个线程, 更新 current_thread
82
83      # 7. 恢复下一个线程的上下文
84      la       t1, current_thread
85      lw       t1, 0(t1)         # t1 = new current_thread
86      lw       sp, 0(t1)         # sp = new_thread->sp (指向保存的上下文)
87
88      # 8. 恢复 mepc 和 mstatus
89      lw       t0, 112(sp)
90      csrwr     mepc, t0
91      lw       t0, 116(sp)
92      csrwr     mstatus, t0
93
94      # 9. 恢复通用寄存器
95      lw       ra, 0(sp)
96      lw       t0, 4(sp)
97      lw       t1, 8(sp)

```

```
98     lw     t2, 12(sp)
99     lw     s0, 16(sp)
100    lw     s1, 20(sp)
101    lw     a0, 24(sp)
102    lw     a1, 28(sp)
103    lw     a2, 32(sp)
104    lw     a3, 36(sp)
105    lw     a4, 40(sp)
106    lw     a5, 44(sp)
107    lw     a6, 48(sp)
108    lw     a7, 52(sp)
109    lw     s2, 56(sp)
110    lw     s3, 60(sp)
111    lw     s4, 64(sp)
112    lw     s5, 68(sp)
113    lw     s6, 72(sp)
114    lw     s7, 76(sp)
115    lw     s8, 80(sp)
116    lw     s9, 84(sp)
117    lw     s10, 88(sp)
118    lw     s11, 92(sp)
119    lw     t3, 96(sp)
120    lw     t4, 100(sp)
121    lw     t5, 104(sp)
122    lw     t6, 108(sp)
123
124    # 10. 释放栈空间
125    addi    sp, sp, 128
126
127    # 11. 设置 mscratch 为调度器栈（供下次使用）
128    la      t0, scheduler_stack
129    csrw    mscratch, t0
130
131    # 12. 返回到新线程
132    mret
```

6.4 mepc 加 4 的问题

关键细节：ECALL 触发陷阱时，mepc 指向 ECALL 指令本身。如果直接 MRET，会重新执行 ECALL，导致无限循环。因此，**必须在保存 mepc 之前或恢复 mepc 之后将其加 4。**

有两种做法：

1. **保存前加 4：**在陷阱入口处，先 `csrr t0, mepc; addi t0, t0, 4; sw t0, 112(sp)`。保存的就是下一条指令的地址，恢复时直接用。
2. **恢复后加 4：**在恢复 mepc 后，`csrr t0, mepc; addi t0, t0, 4; csrw mepc, t0`。

推荐做法一（保存前加 4），因为这样保存的 mepc 就是「恢复后应该执行的地址」，语义更清晰。

修改后的保存代码：

Listing 5: 保存 mepc 时加 4

```

1  # 保存 mepc (加 4, 跳过 ECALL 指令)
2  csrr    t0, mepc
3  addi   t0, t0, 4           # 跳过 ECALL, 指向下一条指令
4  sw     t0, 112(sp)
5  # 保存 mstatus
6  csrr    t0, mstatus
7  sw     t0, 116(sp)
```

7 分情况讨论：上下文切换的各种场景

7.1 场景一：线程 A 首次 yield，切换到线程 B

这是最常见的场景。线程 A 正在运行，调用 `yield()`：

1. A 执行 ECALL，硬件保存 mepc（指向 A 的 ECALL）、mcause=11、mstatus，跳到 `trap_entry`。
2. `trap_entry` 在 A 的栈上保存 30 个通用寄存器 + mepc（已加 4）+ mstatus。
3. 将 A 的 `sp`（指向保存的上下文）写入 `A→sp`。
4. 切换到调度器栈，调用 `schedule()`。
5. `schedule()` 选择线程 B，设置 `current_thread = &B`。
6. 从 `B→sp` 恢复上下文（30 个寄存器 + mepc + mstatus）。
7. 执行 MRET，PC 跳到 B 的 mepc（B 上次 yield 的下一条指令）。

7.2 场景二：线程 B 首次运行（从新创建的线程启动）

新线程从未运行过，没有「上一次 yield 的返回点」。需要**伪造一个栈帧**，使得第一次恢复时看起来像是「从 yield 返回」。

Listing 6: 初始化新线程的栈

```

1 void thread_stack_init(struct thread *t) {
2     uint32_t *sp = (uint32_t *)((uint8_t *)t->stack + t->stack_size);
3     sp = (uint32_t *)((uint32_t)sp & ~0xF); // 16 字节对齐
4
5     sp -= 32; // 分配 128 字节 (32 个字)
6
7     // 伪造寄存器值
8     sp[0] = (uint32_t)thread_entry; // ra ← 线程入口 (MRET 后跳到这里)
9     sp[1] = 0; // t0
10    sp[2] = 0; // t1
11    // ... 其他 t 寄存器为 0 ...
12    sp[4] = 0; // s0
13    // ... 其他 s 寄存器为 0 ...
14    sp[6] = (uint32_t)t->arg; // a0 ← 线程参数
15    sp[7] = 0; // a1
16    // ... 其他 a 寄存器为 0 ...
17
18    // 伪造 mepc 和 mstatus
19    sp[28] = (uint32_t)thread_entry; // mepc ← 线程入口
20    sp[29] = 0x1880; // mstatus: MPP=M, MPIE=1, MIE=1
21
22    t->sp = sp; // 保存栈指针
23 }

```

关键点：新线程的 mepc 设为线程入口函数地址，ra 也设为入口地址（或一个 cleanup 函数）。当 MRET 执行时，PC 跳到 mepc（即线程入口），线程开始运行。mstatus 设为 0x1880（MPP=11, MPIE=1, MIE=1），确保 MRET 后 MIE=1。

7.3 场景三：idle 线程的 yield

RT-Thread 有一个 idle 线程，当没有其他线程就绪时运行。idle 线程通常是一个无限循环调用 yield() 的函数：

Listing 7: idle 线程

```

1 void idle_thread(void *arg) {
2     while (1) {
3         yield(); // 不断 yield, 等待其他线程就绪
4     }
5 }

```

idle 线程的 yield 与普通线程完全相同——保存上下文、调度、恢复。如果调度器发现没有其他就绪线程，会选择 idle 线程自身，相当于「空转」。

7.4 场景四：线程被阻塞后恢复

线程 A 调用 `rt_sem_take()` 但信号量不可用，A 被挂起。此时不执行 `yield` 的 `ECALL`，而是直接调用调度器切换。但底层机制相同——保存 A 的上下文，恢复另一个线程。

Listing 8: 阻塞线程的上下文切换

```

1 void rt_sem_take(semaphore *s) {
2     if (s->count == 0) {
3         // 信号量不可用，阻塞当前线程
4         current_thread->state = BLOCKED;
5         // 将当前线程加入信号量等待队列
6         list_add(&s->wait_list, &current_thread->list);
7         // 触发调度 (底层也是 ECALL yield)
8         yield();
9         // 当线程被唤醒时，从这里继续执行
10    }
11    s->count--;
12 }

```

7.5 场景五：中断嵌套（如果有硬件中断）

如果处理器实现了硬件中断（如 CLINT 定时器），中断可能在 `yield` 陷阱处理过程中发生。此时 `mstatus.MIE=0`（陷阱进入时硬件清零），中断被屏蔽，不会嵌套。陷阱处理完成后 `MRET` 恢复 `MIE=1`，中断才能响应。

如果需要支持中断嵌套：在陷阱处理程序中手动重新开启 `MIE` (`csrs mstatus, MIE_BIT`)，允许更高优先级中断打断。但这需要更复杂的栈管理（每个中断级别需要独立栈），RT-Thread `yield` 方案通常不需要。

8 yield 的完整实现代码

8.1 汇编入口（trap_entry）

以下是经过优化的完整陷阱入口代码，使用循环减少代码量：

Listing 9: 优化后的 trap_entry

```

1     .section .text
2     .globl trap_entry
3     .align 2
4 trap_entry:
5     # 保存上下文 (30 个通用寄存器 + mepc + mstatus = 32 个字)
6     addi    sp, sp, -128
7
8     # 保存 ra (x1)
9     sw      ra, 0(sp)

```

```

10
11 # 保存 t0-t6 (x5-x7, x28-x31)
12 SW      t0, 4(sp)
13 SW      t1, 8(sp)
14 SW      t2, 12(sp)
15 SW      t3, 16(sp)
16 SW      t4, 20(sp)
17 SW      t5, 24(sp)
18 SW      t6, 28(sp)
19
20 # 保存 s0-s11 (x8-x9, x18-x27)
21 SW      s0, 32(sp)
22 SW      s1, 36(sp)
23 SW      s2, 40(sp)
24 SW      s3, 44(sp)
25 SW      s4, 48(sp)
26 SW      s5, 52(sp)
27 SW      s6, 56(sp)
28 SW      s7, 60(sp)
29 SW      s8, 64(sp)
30 SW      s9, 68(sp)
31 SW      s10, 72(sp)
32 SW      s11, 76(sp)
33
34 # 保存 a0-a7 (x10-x17)
35 SW      a0, 80(sp)
36 SW      a1, 84(sp)
37 SW      a2, 88(sp)
38 SW      a3, 92(sp)
39 SW      a4, 96(sp)
40 SW      a5, 100(sp)
41 SW      a6, 104(sp)
42 SW      a7, 108(sp)
43
44 # 保存 mepc (加 4 跳过 ECALL) 和 mstatus
45 csrr    t0, mepc
46 addi    t0, t0, 4
47 SW      t0, 112(sp)
48 csrr    t0, mstatus
49 SW      t0, 116(sp)
50
51 # 保存当前 sp 到 TCB
52 la      t0, current_thread
53 lw      t0, 0(t0)
54 SW      sp, 0(t0)      # current_thread->sp = sp

```

```

55
56 # 切换到调度器栈
57 la      sp, scheduler_stack_top
58
59 # 调用 C 调度器
60 call    schedule
61
62 # 恢复新线程的 sp
63 la      t0, current_thread
64 lw      t0, 0(t0)
65 lw      sp, 0(t0)      # sp = current_thread->sp
66
67 # 恢复 mepc 和 mstatus
68 lw      t0, 112(sp)
69 csrw    mepc, t0
70 lw      t0, 116(sp)
71 csrw    mstatus, t0
72
73 # 恢复通用寄存器
74 lw      ra, 0(sp)
75 lw      t0, 4(sp)
76 lw      t1, 8(sp)
77 lw      t2, 12(sp)
78 lw      s0, 16(sp)
79 lw      s1, 20(sp)
80 lw      a0, 24(sp)
81 lw      a1, 28(sp)
82 lw      a2, 32(sp)
83 lw      a3, 36(sp)
84 lw      a4, 40(sp)
85 lw      a5, 44(sp)
86 lw      a6, 48(sp)
87 lw      a7, 52(sp)
88 lw      s2, 56(sp)
89 lw      s3, 60(sp)
90 lw      s4, 64(sp)
91 lw      s5, 68(sp)
92 lw      s6, 72(sp)
93 lw      s7, 76(sp)
94 lw      s8, 80(sp)
95 lw      s9, 84(sp)
96 lw      s10, 88(sp)
97 lw      s11, 92(sp)
98 lw      t3, 96(sp)
99 lw      t4, 100(sp)

```

```

100    lw      t5, 104(sp)
101    lw      t6, 108(sp)
102
103    addi    sp, sp, 128
104    mret

```

8.2 C 语言调度器

Listing 10: C 语言调度器

```

1  struct thread *current_thread;
2
3  void schedule(void) {
4      // 简单的轮转调度
5      struct thread *prev = current_thread;
6      struct thread *next = pick_next_thread();
7
8      if (next == prev) {
9          return; // 没有切换
10     }
11
12     prev->state = READY;
13     next->state = RUNNING;
14     current_thread = next;
15 }
16
17 struct thread *pick_next_thread(void) {
18     // 遍历线程列表, 找到下一个 READY 线程
19     for (int i = 0; i < num_threads; i++) {
20         int idx = (current_thread_index + 1 + i) % num_threads;
21         if (threads[idx].state == READY) {
22             current_thread_index = idx;
23             return &threads[idx];
24         }
25     }
26     // 没有就绪线程, 返回 idle
27     return &idle_thread;
28 }

```

8.3 yield 函数

Listing 11: yield 函数

```

1  void yield(void) {
2      // ECALL 触发 M 模式陷阱
3      asm volatile("ecall");

```

```
4 }
```

8.4 线程启动

Listing 12: 线程启动与清理

```
1 void thread_entry_wrapper(void *arg) {
2     struct thread *t = current_thread;
3     t->entry(t->arg);          // 调用线程入口函数
4     // 线程返回后, 标记为 FINISHED
5     t->state = FINISHED;
6     yield();                  // 切换到其他线程
7     // 不应该到达这里
8     while (1);
9 }
```

9 常见陷阱与调试技巧

9.1 陷阱一：忘记 mepc 加 4

症状：系统在 yield 后立即再次 yield，形成无限循环。

原因：mepc 指向 ECALL 指令本身，MRET 后重新执行 ECALL。

修复：在保存 mepc 时加 4。

9.2 陷阱二：mscratch 使用错误

症状：第一次 yield 正常，第二次 yield 崩溃。

原因：mscratch 在第一次 yield 后被设为线程 A 的 sp，第二次 yield 时 csrrw sp, mscratch, sp 交换了错误的值。

修复：不使用 mscratch 交换，直接在 trap_entry 中用固定地址的调度器栈。或者正确管理 mscratch 的值。

9.3 陷阱三：栈对齐问题

症状：某些指令（如 lw）触发地址对齐异常。

原因：RISC-V 要求 sp 16 字节对齐（ABI 规定）。如果栈没有对齐，sw/lw 访问保存的上下文时可能不对齐。

修复：初始化栈时对齐到 16 字节：sp = (sp & ~0xF)。

9.4 陷阱四：gp 寄存器未正确设置

症状：链接器生成的全局变量访问异常。

原因：gp (x3) 是全球指针寄存器，用于高效访问全局变量。如果线程切换后 gp 被破坏，全局变量访问会出错。

修复：不保存/恢复 gp（所有线程共享同一个 gp 值），确保 trap_entry 不修改 gp。

9.5 陷阱五：调度器栈溢出

症状：调度器执行 `schedule()` 时崩溃。

原因：调度器栈太小，`schedule()` 函数的局部变量溢出。

修复：给调度器栈分配足够大的空间（至少 4KB）。

9.6 调试技巧

1. 在 `trap_entry` 中输出 `mcause` 和 `mepc`：确认陷阱原因和来源。
2. 检查 `sp` 的值：确认 `sp` 在保存/恢复前后一致。
3. 检查 `TCB->sp`：确认保存的 `sp` 指向正确的上下文区域。
4. 单步执行 `MRET`：确认 `MRET` 后 `PC` 跳到正确的地址。
5. 使用 `spike` 或 `QEMU` 对比：在模拟器上跑同样的代码，对比行为。

Part III

RT-Thread 移植实践

本部分将前两部分的理论落地为可操作的 RT-Thread 移植指南。我们以 RT-Thread Nano 为基础，讲解如何在纯 M 模式下使用 `yield` 软中断实现线程调度，覆盖从启动到第一个线程运行的完整流程。

10 RT-Thread 的架构与移植要点

10.1 RT-Thread Nano 的组件

RT-Thread Nano 是 RT-Thread 的精简版，包含以下核心组件：

- **线程管理：**线程创建、删除、挂起、恢复。
- **调度器：**优先级调度，同优先级时间片轮转。
- **IPC：**信号量、互斥量、邮箱、消息队列。
- **定时器：**软件定时器（需要硬件 tick 提供）。
- **内存管理：**小内存分配器（`mem.c`）或 `slab` 分配器。

移植 RT-Thread Nano 需要实现的接口：

接口说明

<code>rt_hw_context_switch()</code>	线程切换（ <code>yield</code> 的底层）
<code>rt_hw_context_switch_to()</code>	切换到指定线程（首次启动）
<code>rt_hw_context_switch_interrupt()</code>	中断中切换（ <code>yield</code> 方案可复用普通切换）
<code>rt_hw_stack_init()</code>	初始化线程栈
<code>rt_hw_exception()</code>	异常处理（非法指令等）

10.2 yield 方案与标准方案的区别

特性标准方案（定时器中断）yield 方案（软中断）		
调度触发	硬件定时器中断（1ms）	线程主动调用 yield()
抢占式	是（中断可打断任何线程）	否（协作式，线程必须主动 yield）
时间片	有（定时器 tick 计数）	无（线程运行到 yield 为止）
硬件依赖	CLINT 定时器中断	仅需 ECALL + MRET
延迟	确定性（ $\leq 1\text{ms}$ ）	不确定（取决于线程何时 yield）

yield 方案的局限性：如果一个线程不调用 yield()（如死循环），其他线程永远无法运行。因此 RT-Thread 的所有线程必须频繁调用 **yield**（或调用会 yield 的 API，如 rt_thread_delay、rt_sem_take）。idle 线程本身就是一个不断 yield 的循环。

11 移植的完整步骤

11.1 步骤一：实现 rt_hw_stack_init

RT-Thread 创建线程时调用此函数初始化栈：

Listing 13: rt_hw_stack_init 实现

```

1  rt_uint8_t *rt_hw_stack_init(void *entry, void *arg,
2                                void *stack_start, rt_uint32_t stack_size) {
3      rt_uint32_t *sp;
4      rt_uint32_t *stk;
5
6      // 计算栈顶（高地址），16 字节对齐
7      sp = (rt_uint32_t *)((rt_uint8_t *)stack_start + stack_size);
8      sp = (rt_uint32_t *)((rt_uint32_t)sp & ~0xF);
9
10     // 分配 32 个字的上下文空间
11     sp -= 32;
12
13     // 初始化寄存器（索引对应 part2 中的栈帧布局）
14     sp[0] = (rt_uint32_t)entry;    // ra: 线程入口
15     sp[1] = 0;                    // t0
16     sp[2] = 0;                    // t1
17     sp[3] = 0;                    // t2

```

```

18     sp[4] = 0;                // s0
19     sp[5] = 0;                // s1
20     sp[6] = (rt_uint32_t)arg; // a0: 线程参数
21     sp[7] = 0;                // a1
22     sp[8] = 0;                // a2
23     sp[9] = 0;                // a3
24     sp[10] = 0;               // a4
25     sp[11] = 0;               // a5
26     sp[12] = 0;               // a6
27     sp[13] = 0;               // a7
28     sp[14] = 0;               // s2
29     sp[15] = 0;               // s3
30     sp[16] = 0;               // s4
31     sp[17] = 0;               // s5
32     sp[18] = 0;               // s6
33     sp[19] = 0;               // s7
34     sp[20] = 0;               // s8
35     sp[21] = 0;               // s9
36     sp[22] = 0;               // s10
37     sp[23] = 0;               // s11
38     sp[24] = 0;               // t3
39     sp[25] = 0;               // t4
40     sp[26] = 0;               // t5
41     sp[27] = 0;               // t6
42
43     // mepc: 线程入口地址
44     sp[28] = (rt_uint32_t)entry;
45     // mstatus: MPP=M(11), MPIE=1, MIE=1
46     sp[29] = 0x1880;
47
48     // 返回栈指针（指向上下文的最低地址）
49     return (rt_uint8_t *)sp;
50 }

```

11.2 步骤二：实现 rt_hw_context_switch

这是 yield 的核心——保存当前线程上下文，切换到目标线程：

Listing 14: rt_hw_context_switch（汇编）

```

1     .globl rt_hw_context_switch
2 rt_hw_context_switch:
3     // 保存当前线程上下文
4     addi    sp, sp, -128
5
6     sw      ra,    0(sp)

```

```

7      SW      t0,    4(sp)
8      SW      t1,    8(sp)
9      SW      t2,   12(sp)
10     SW      s0,   16(sp)
11     SW      s1,   20(sp)
12     SW      a0,   24(sp)
13     SW      a1,   28(sp)
14     SW      a2,   32(sp)
15     SW      a3,   36(sp)
16     SW      a4,   40(sp)
17     SW      a5,   44(sp)
18     SW      a6,   48(sp)
19     SW      a7,   52(sp)
20     SW      s2,   56(sp)
21     SW      s3,   60(sp)
22     SW      s4,   64(sp)
23     SW      s5,   68(sp)
24     SW      s6,   72(sp)
25     SW      s7,   76(sp)
26     SW      s8,   80(sp)
27     SW      s9,   84(sp)
28     SW      s10, 88(sp)
29     SW      s11, 92(sp)
30     SW      t3,   96(sp)
31     SW      t4,  100(sp)
32     SW      t5,  104(sp)
33     SW      t6,  108(sp)
34
35     // 保存 mepc (加 4) 和 mstatus
36     csrr     t0, mepc
37     addi     t0, t0, 4
38     SW      t0, 112(sp)
39     csrr     t0, mstatus
40     SW      t0, 116(sp)
41
42     // 保存 sp 到 from_thread->sp
43     SW      sp, 0(a0)      // a0 = from_thread (rt_thread)
44
45     // 从 to_thread->sp 恢复
46     lw      sp, 0(a1)      // a1 = to_thread
47
48     // 恢复 mepc 和 mstatus
49     lw      t0, 112(sp)
50     csrwr   mepc, t0
51     lw      t0, 116(sp)

```

```

52     csrw    mstatus, t0
53
54     // 恢复通用寄存器
55     lw      ra,    0(sp)
56     lw      t0,    4(sp)
57     lw      t1,    8(sp)
58     lw      t2,   12(sp)
59     lw      s0,   16(sp)
60     lw      s1,   20(sp)
61     lw      a0,   24(sp)
62     lw      a1,   28(sp)
63     lw      a2,   32(sp)
64     lw      a3,   36(sp)
65     lw      a4,   40(sp)
66     lw      a5,   44(sp)
67     lw      a6,   48(sp)
68     lw      a7,   52(sp)
69     lw      s2,   56(sp)
70     lw      s3,   60(sp)
71     lw      s4,   64(sp)
72     lw      s5,   68(sp)
73     lw      s6,   72(sp)
74     lw      s7,   76(sp)
75     lw      s8,   80(sp)
76     lw      s9,   84(sp)
77     lw      s10,  88(sp)
78     lw      s11,  92(sp)
79     lw      t3,   96(sp)
80     lw      t4,  100(sp)
81     lw      t5,  104(sp)
82     lw      t6,  108(sp)
83
84     addi    sp, sp, 128
85     mret

```

11.3 步骤三：实现 `rt_hw_context_switch_to`

用于首次启动线程（没有「from 线程」）：

Listing 15: `rt_hw_context_switch_to`

```

1     .globl rt_hw_context_switch_to
2     rt_hw_context_switch_to:
3         // a0 = to_thread
4         lw      sp, 0(a0)      // 加载目标线程的 sp
5

```

```

6    // 恢复 mepc 和 mstatus
7    lw      t0, 112(sp)
8    csrw    mepc, t0
9    lw      t0, 116(sp)
10   csrw    mstatus, t0
11
12   // 恢复通用寄存器（同上，省略）
13
14   addi     sp, sp, 128
15   mret                                // 跳到线程入口

```

11.4 步骤四：实现 rt_hw_exception

处理非 ECALL 异常（非法指令、地址错误等）：

Listing 16: rt_hw_exception

```

1 void rt_hw_exception(rt_uint32_t mcause, rt_uint32_t mepc) {
2     rt_uint32_t cause = mcause & 0x7FFFFFFF;
3     rt_uint32_t is_interrupt = (mcause >> 31) & 1;
4
5     if (is_interrupt) {
6         // 中断处理（如果有硬件中断）
7         // ...
8     } else {
9         // 异常处理
10        switch (cause) {
11            case 2: // 非法指令
12                rt_kprintf("Illegal instruction at 0x%08x\n", mepc);
13                break;
14            case 4: // Load 地址对齐错误
15                rt_kprintf("Load address misaligned at 0x%08x\n", mepc);
16                break;
17            case 6: // Store 地址对齐错误
18                rt_kprintf("Store address misaligned at 0x%08x\n", mepc);
19                break;
20            case 11: // ECALL from M-mode
21                // yield, 不需要处理（已在 trap_entry 中处理）
22                break;
23            default:
24                rt_kprintf("Unknown exception %d at 0x%08x\n", cause, mepc);
25                break;
26        }
27    }
28
29    // 对于致命异常，可以终止线程或停机

```

```

30     while (1);
31 }

```

11.5 步骤五：连接到 RT-Thread 调度器

RT-Thread 的调度器在 `rt_schedule()` 中调用 `rt_hw_context_switch()`。在 `yield` 方案中, `rt_schedule()` 的调用链:

1. 线程调用 `rt_thread_yield()` 或 `rt_thread_delay()`。
2. RT-Thread 调用 `rt_schedule()`。
3. `rt_schedule()` 选择下一个线程, 调用 `rt_hw_context_switch(from, to)`。
4. `rt_hw_context_switch` 执行 ECALL (或直接执行保存/恢复逻辑)。

两种实现方式:

1. **ECALL 方式:** `rt_hw_context_switch` 只执行 ECALL, 陷阱处理程序完成保存/恢复。优点是统一入口; 缺点是 `from/to` 参数需要通过全局变量传递。
2. **直接切换方式:** `rt_hw_context_switch` 直接执行保存/恢复逻辑 (不经过 ECALL)。优点是参数通过 `a0/a1` 传递, 效率高; 缺点是需要手动管理 `mepc` (设为返回地址)。

RT-Thread 官方 RISC-V 移植使用方式二。本文档的代码示例也是方式二。

12 从启动到第一个线程运行

12.1 启动流程

Listing 17: 启动代码

```

1     .section .text.init
2     .globl _start
3 _start:
4     # 设置栈指针 (启动栈)
5     la      sp, _stack_top
6
7     # 设置 mtvec (陷阱入口)
8     la      t0, trap_entry
9     csrw    mtvec, t0
10
11    # 设置 gp (全局指针)
12    .option push
13    .option norelax
14    la      gp, __global_pointer$
15    .option pop
16
17    # 清零 BSS 段

```

```

18     la      t0, _bss_start
19     la      t1, _bss_end
20 1:    bgeu   t0, t1, 2f
21     sw      zero, 0(t0)
22     addi    t0, t0, 4
23     j       1b
24 2:
25
26     # 调用 C 入口
27     call    rtthread_startup
28
29     # 不应该返回
30 3:    wfi
31     j       3b

```

12.2 rtthread_startup

Listing 18: rtthread_startup

```

1  int rtthread_startup(void) {
2      // 初始化 RT-Thread 内核
3      rt_hw_interrupt_disable();
4      rt_show_version();           // 打印版本信息
5      rt_system_timer_init();      // 初始化定时器
6      rt_system_scheduler_init();  // 初始化调度器
7
8      // 创建主线程
9      rt_application_init();
10
11     // 创建 idle 线程
12     rt_thread_idle_init();
13
14     // 启动调度器（永不返回）
15     rt_system_scheduler_start();
16
17     return 0;
18 }

```

12.3 rt_system_scheduler_start

这是启动第一个线程的函数，调用 `rt_hw_context_switch_to`：

Listing 19: 启动第一个线程

```

1  void rt_system_scheduler_start(void) {
2      struct rt_thread *to_thread;
3

```

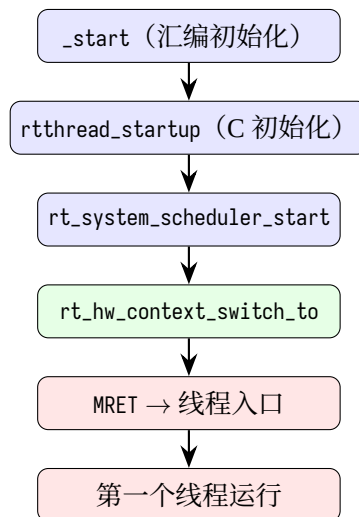
```

4 // 选择第一个线程
5 to_thread = rt_list_entry(rt_thread_priority_table[0].next,
6                          struct rt_thread, tlist);
7
8 // 设置当前线程
9 rt_current_thread = to_thread;
10
11 // 切换到第一个线程 (永不返回)
12 rt_hw_context_switch_to((rt_uint32_t)&to_thread->sp);
13 }

```

12.4 第一个线程的运行

`rt_hw_context_switch_to` 从线程的栈恢复伪造的上下文, 执行 MRET。MRET 将 PC 设为 `mepc` (线程入口函数地址), 线程开始运行。



13 进阶：从 yield 到定时器中断调度

13.1 为什么需要定时器中断

`yield` 方案是协作式调度, 线程必须主动 `yield`。如果一个线程长时间不 `yield` (如计算密集型任务), 其他线程无法运行。定时器中断可以实现**抢占式调度**——定时器每 1ms 触发中断, 中断处理程序调用调度器, 强制切换线程。

13.2 实现 CLINT 定时器中断

Listing 20: CLINT 定时器初始化

```

1 #define CLINT_BASE      0x02000000
2 #define CLINT_MTIMECMP (CLINT_BASE + 0x4000)
3 #define CLINT_MTIME     (CLINT_BASE + 0xBFF8)
4
5 void rt_hw_timer_init(void) {

```

```

6    // 读取当前 mtime
7    uint64_t now = *(uint64_t *)CLINT_MTIME;
8    // 设置下次中断时间 (1ms 后)
9    *(uint64_t *)CLINT_MTIMECMP = now + TIMER_INTERVAL;
10   // 使能 M 级定时器中断
11   csr_set(mie, MIE_MTIE);
12   // 使能全局中断
13   csr_set(mstatus, MSTATUS_MIE);
14 }

```

13.3 定时器中断处理

Listing 21: 定时器中断处理

```

1  trap_entry:
2      # 保存上下文 (同 yield 方案)
3      addi    sp, sp, -128
4      # ... 保存所有寄存器 ...
5
6      # 读取 mcause 判断是中断还是异常
7      csrr    t0, mcause
8      li      t1, 0x80000000
9      and     t0, t0, t1
10     beqz    t0, handle_exception # 异常
11
12     # 是中断, 读取 cause
13     csrr    t0, mcause
14     and     t0, t0, 0x7FFFFFFF
15     li      t1, 7                # MTI cause=7
16     beq     t0, t1, handle_timer # 定时器中断
17
18  handle_timer:
19     # 重设 mtimecmp
20     la      t0, CLINT_MTIME
21     ld      t0, 0(t0)
22     li      t1, TIMER_INTERVAL
23     add     t0, t0, t1
24     la      t1, CLINT_MTIMECMP
25     sd      t0, 0(t1)
26
27     # 调用 RT-Thread tick 处理
28     call    rt_timer_tick
29
30     # 调用调度器
31     call    rt_schedule

```

```

32
33     # 恢复上下文 (同 yield 方案)
34     # ... 恢复所有寄存器 ...
35     addi    sp, sp, 128
36     mret

```

13.4 yield 与定时器中断的共存

在实现了定时器中断后，yield 仍然可以使用——线程主动 yield 时走 ECALL 路径，定时器中断走中断路径。两者共用同一个 trap_entry，通过 mcause 最高位区分（中断 =1，异常 =0）。

关键区别：定时器中断时 mepc 指向下一条指令（中断在指令之间产生），不需要加 4。ECALL 异常时 mepc 指向 ECALL 指令本身，需要加 4。陷阱处理程序必须根据 mcause 最高位区分这两种情况。

Listing 22: 区分中断与异常的 mepc 处理

```

1      csrr    t0, mcause
2      li      t1, 0x80000000
3      and     t0, t0, t1
4      bnez    t0, save_mepc_no_add # 中断: mepc 不加 4
5
6      # 异常: mepc 加 4
7      csrr    t0, mepc
8      addi    t0, t0, 4
9      j       save_mepc
10
11 save_mepc_no_add:
12     csrr    t0, mepc
13
14 save_mepc:
15     sw      t0, 112(sp)

```

14 完整示例：两个线程交替运行

14.1 main.c

Listing 23: 两个线程交替运行

```

1 #include <rtthread.h>
2
3 void thread1_entry(void *arg) {
4     while (1) {
5         rt_kprintf("Thread 1 running\n");
6         rt_thread_yield(); // 主动让出 CPU
7     }

```

```

8 }
9
10 void thread2_entry(void *arg) {
11     while (1) {
12         rt_kprintf("Thread 2 running\n");
13         rt_thread_yield();
14     }
15 }
16
17 int main(void) {
18     rt_thread_t tid1, tid2;
19
20     tid1 = rt_thread_create("thread1", thread1_entry, RT_NULL,
21                             1024, 10, 10);
22     tid2 = rt_thread_create("thread2", thread2_entry, RT_NULL,
23                             1024, 10, 10);
24
25     if (tid1) rt_thread_startup(tid1);
26     if (tid2) rt_thread_startup(tid2);
27
28     return 0;
29 }

```

14.2 执行流程

1. _start 初始化硬件，调用 rtthread_startup。
2. rtthread_startup 初始化内核，创建 main 线程和 idle 线程。
3. main 线程创建 thread1 和 thread2，调用 rt_thread_startup 将它们加入就绪队列。
4. main 线程返回后，idle 线程运行，调用 yield。
5. 调度器选择 thread1，rt_hw_context_switch_to 恢复 thread1 的上下文。
6. thread1 打印“Thread 1 running”，调用 rt_thread_yield。
7. rt_hw_context_switch 保存 thread1 上下文，调度器选择 thread2。
8. thread2 打印“Thread 2 running”，调用 rt_thread_yield。
9. 调度器选择 thread1，恢复 thread1 上下文，从上次 yield 的下一条指令继续。
10. 循环 6–9，两个线程交替运行。

14.3 验证方法

1. 在 QEMU 或 spike 上运行，观察串口输出是否交替出现“Thread 1”和“Thread 2”。
2. 在自己的处理器上运行，观察串口输出。
3. 如果输出卡住，检查 trap_entry 的保存/恢复逻辑。

4. 如果输出乱码，检查 UART 驱动和 `rt_kprintf` 实现。
5. 如果线程不切换，检查 `rt_schedule` 是否正确调用了 `rt_hw_context_switch`。

最终建议：先在 QEMU 上验证 RT-Thread 的 `yield` 方案能正常工作，再移植到你自己的处理器。QEMU 提供了可靠的参考行为，可以帮助你区分「软件 bug」和「硬件 bug」。如果 QEMU 上正常但你的处理器上异常，问题大概率在硬件的 CSR 实现、ECALL/MRET 行为、或内存访问对齐上。